Workshop: From Zero to ASIC Hero! - Introductory Hands-on: HEEPup!

Part #1: Booting a RISC-V SoC with X-HEEP

July 07, 2026

1. Introduction

This lab introduces students to the **X-HEEP** platform, a lightweight and extensible RISC-V SoC template designed for **low-power edge computing applications**. The lab will cover setting up the development environment, running a simple "Hello World" application using simulation, exploring the main **X-HEEP hierarchy**, and understanding the role of **Makefiles, automation scripts, and FuseSoC** in managing the build process.

All the designs from this course will be made using **SystemVerilog**, a hardware description and hardware verification language standardized by the Institute of Electrical and Electronics Engineers (IEEE) as IEEE 1800-2023 [IEEE'24], which has become the de-facto standard in the open-hardware ecosystem. In particular, the code will be written according to the guidelines released by lowRISC.[1]

This teaching material has been prepared using information coming from the X-HEEP framework (see https://github.com/x-heep and https://github.com/x-heep/GR-HEEP).

2. Set-up environment

The main architecture of the framework is depicted in Figure 1. The main elements are as following:

[1] https://github.com/lowRISC/style-guides/blob/master/VerilogCodingStyle.md

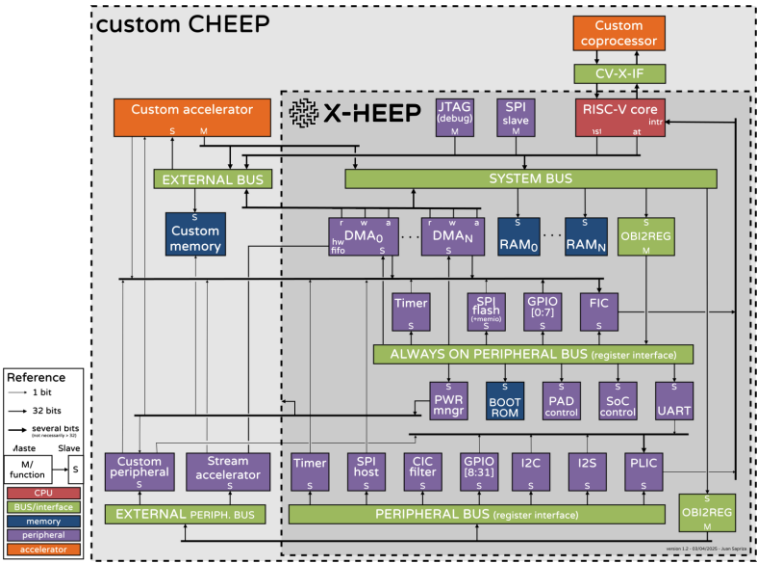


Figure 1. X-HEEP framework architecture

Today, we are going to focus on the X-HEEP part only. Thus, we will try now to set-up all the environment to be able to simulate the platform. Note that all the dependencies and initial tools are already installed into the PCs of the lab. They include everything described in https://x-heep.readthedocs.io/en/latest/GettingStarted/Setup.html.

- Open a terminal and clone the repository:

```
(base) jose@jose-PC:~/UPM/uCredentials$ git clone https://github.com/x-heep/GR-HEEP
Cloning into 'GR-HEEP'...
remote: Enumerating objects: 4464, done.
remote: Counting objects: 100% (940/940), done.
remote: Compressing objects: 100% (735/735), done.
remote: Total 4464 (delta 256), reused 547 (delta 164), pack-reused 3524 (from 1)
Receiving objects: 100% (4464/4464), 59.67 MiB | 34.23 MiB/s, done.
Resolving deltas: 100% (1268/1268), done.
(base) jose@jose-PC:~/UPM/uCredentials$
```

- Activate the conda environment, install some dependencies and export the needed paths for the whole Lab:

```
conda activate core-v-mini-mcu && \
python -m pip install tabulate black &&\
export PATH=/home/$USER/tools/verilator/5.040/bin:/home/$USER/tools/riscv/bin:$PATH && \
export RISCV_XHEEP=/home/$USER/tools/riscv && \
export COMPILER_PREFIX=riscv32-unknown- && \
export CCACHE_DISABLE=1 && \
export VERIBLE_VERSION=v0.0-4023-gc1271a00 && \
export PATH=/home/$USER/tools/verible/${VERIBLE_VERSION}/bin:$PATH && \
make -C hw/vendor/x-heep/hw/ip/boot_rom all
```

- Open VSCode to see the whole framework

```
cd GR-HEEP &&\
code .
```

At this point you are ready to start creating X-HEEP based SoCs and simulate them. Before that, let's try to understand different key elements.

3. Vendor, Main Makefile, automatization engines (mcu_gen.py) and Fusesoc

a) In X-HEEP we use the vendor tool from OpenTitan to manage the code which we include from external repositories. The vendor tool is a simple tool that allows you to copy upstream sources into the repository. You can find the vendor tool in ***util/vendor.py***, and the vendored repositories in *hw/vendor*.
In *hw/vendor* you will find a ***<organization>_<repo_name>.vendor.hjson*** file for each vendored repository. This vendor description file contains the information needed by the vendor tool to fetch the upstream repository and perform any necessary modifications to it.

b) The Makefile in X-HEEP is designed to streamline and automate the compilation, simulation, and FPGA synthesis processes. It provides a user-friendly interface for managing the entire SoC development flow, enabling easy execution of common tasks.
Key Features:
- Application Compilation: Automatically compile RISC-V software applications.
- Simulation Execution: Calls Verilator for cycle-accurate simulation.
- FPGA Synthesis: Supports FPGA builds using Vivado.
- Dependency Management: Ensures all required components are correctly built.

c) The **mcu_gen.py** script is an automation engine that dynamically configures the SoC memory map and peripheral interfaces. It simplifies hardware/software integration by: 1) Automatically generating HDL and C headers (for memory-mapped registers) from tpl files, 2) Ensuring consistency between hardware and software components, 3) Generating linker scripts based on the system's memory structure.

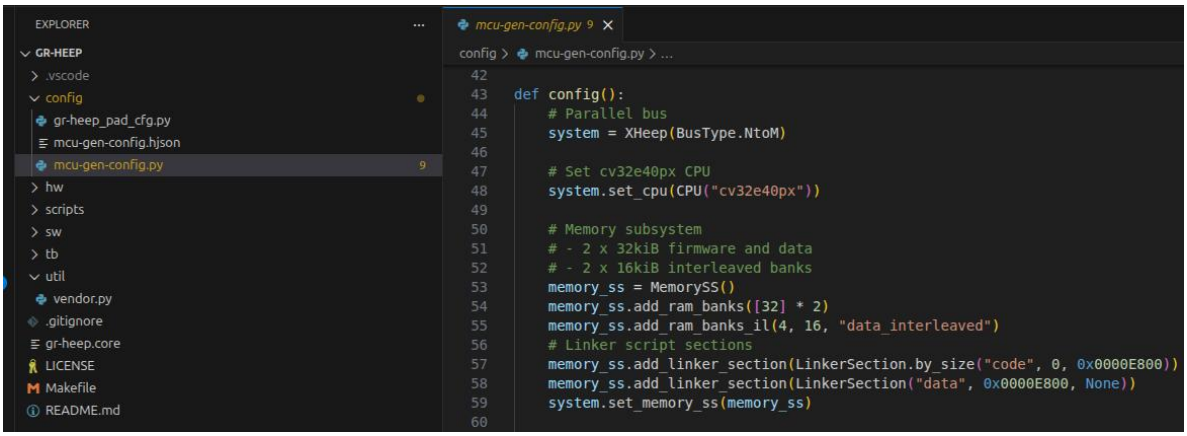
d) **FuseSoC** is an open-source package manager and build system for hardware design, primarily focusing on FPGA and ASIC development. It simplifies working with complex hardware projects by managing dependencies, configurations, and tool flows. It is the tool we use in X-HEEP to manage the different flows that we support. The main building blocks of FuseSoC are the ***.core file***s. A *.core file* is self-contained description of a hardware module, including its sources, dependencies, and configurations. You can find the .core files for X-HEEP all around the project, especially in the hw directory and its subdirectories. However, the main .core file is core-v-mini-mcu.core. To understand .core files, modify them, or write your own, please check the documentation on this in the FuseSoC documentation. X-HEEP's Makefile uses FuseSoC behind the scenes to build, simulate, and synthesize the design, among other uses.

After knowing these tools, we can issue the following command

```
make mcu-gen
```

```
(core-v-mini-mcu) jose@jose-PC:~/Desktop/CSSE_Labs/Lab2/GR-HEEP$ make mcu-gen
make -C hw/vendor/xheep/spi_reg SW_DIR=/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep/sw/device/lib/drivers/
make[3]: Entering directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep/hw/vendor/xheep/spi'
Generating w25q128jw controller registers...
make[3]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep/hw/vendor/xheep/spi'
make verible
make[3]: Entering directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep'
$You are executing from: /home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep
USING MINICONDA core-v-mini-mcu
util/format-verible;
make[3]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep'
make format-python
make[3]: Entering directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep'
$You are executing from: /home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep
USING MINICONDA core-v-mini-mcu
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black util/xheep_gen
All done! ✨ 🍰 ✨
56 files left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black util/periph_structs_gen
All done! ✨ 🍰 ✨
1 file left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black util/waiver-gen.py
All done! ✨ 🍰 ✨
1 file left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black util/c_gen.py
All done! ✨ 🍰 ✨
1 file left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black test/test_x_heep_gen
All done! ✨ 🍰 ✨
5 files left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black test/test_apps
All done! ✨ 🍰 ✨
6 files left unchanged.
/home/jose/miniconda3/envs/core-v-mini-mcu/bin/python -m black configs
All done! ✨ 🍰 ✨
5 files left unchanged.
make[3]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep'
make[2]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP/hw/vendor/x-heep'
make[1]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP'
make verible
make[1]: Entering directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP'
USING MINICONDA core-v-mini-mcu
hw/gr-heep/gr_heep_bus.sv:139:9-22: Do not use $test$plusargs to access plusargs, use $value$plusargs instead. [Style: plusarg-value-assignment] [plusarg-assignment]
make[1]: Leaving directory '/home/jose/Desktop/CSSE_Labs/Lab2/GR-HEEP'
✅ DONE! X-HEEP MCU and GR-HEEP generated successfully
(core-v-mini-mcu) jose@jose-PC:~/Desktop/CSSE_Labs/Lab2/GR-HEEP$
```

This will generate the basic architecture consisting on 2 memory banks of 32KB and the cv32e20 with the configuration stated in:



You do not need this for the lab, but…the easiest way of changing the default values is with some arguments. For example, you can change the CPU type (cv32e20), the default bus type (NtoM), the default continuous memory size (2 banks), or the default interleaved memory size (4 banks) with:

make mcu-gen CPU=cv32e40p BUS=NtoM MEMORY_BANKS=12 MEMORY_BANKS_IL=4

This generates X-HEEP with the cv32e40p core, a parallel bus, and 16 memory banks (12 continuous and 4 interleaved), 32KB each, for a total memory of 512KB.

Note that after performing the *make mcu-gen …* command(s), **the different .sv file from the tpl.sv files are created as well as the headers in the HAL of the peripherals containing the memory mapped structures (to easily access in the code to the different registers or bitfields).**

4. Simulating “hello world” App with Verilator and checking signals with GTKWave

Verilator is an open-source Verilog and SystemVerilog simulator that compiles into multithreaded C++ or SystemC code.[2] It is well-known and has been widely adopted by both community and industry, as many open-source hardware projects rely on it as their main simulation infrastructure.

After having generated the system with the required amount of memory, it is time to simulate using Verilator.

```
make verilator-build
```

This will create the build folder in root, where the model has been compiled. Then, we can just run the app on it:

```
make verilator-run-app
```

In case you are facing problems with the zicsr due to an older riscv toolchain, you can just modify the by-default ARCH parameter into the vendorised x-heep makefile:

```
ARCH   ?= rv32imc
```

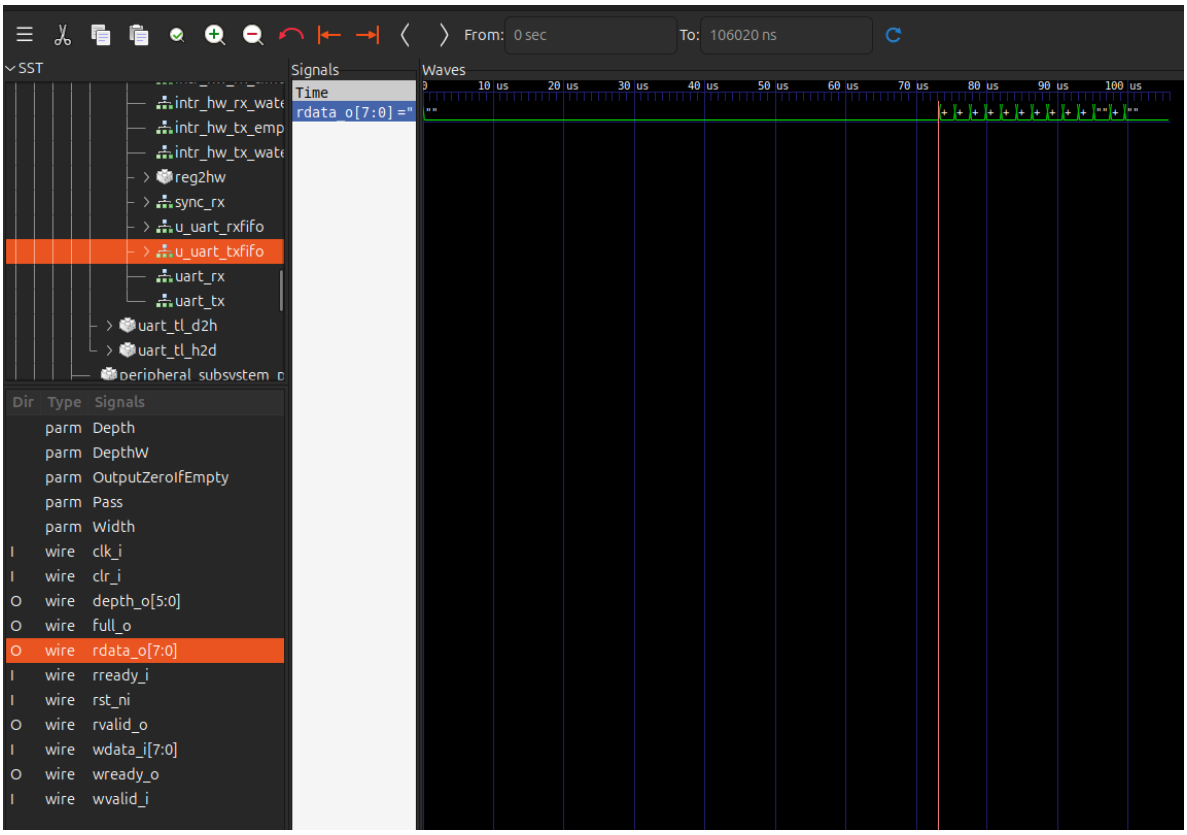
```
get=sim --tool=verilator  \
        --run x-heep:systems:gr-heep  \
        --run_options="+firmware=../../../sw/build/main.hex "
WARNING: Unknown item cache_type in section Generators
WARNING: Unknown item file_input_parameters in section Generators
WARNING: Unknown item include_dirs in section Fileset
WARNING: Unknown item formal in section Target
WARNING: Unknown item formal in section Target
WARNING: Unknown item formal in section Target
INFO: Running
INFO: Running pre_run script remove_uart_log
INFO: Running simulation
[TESTBENCH]: No OpenOCD is used
[TESTBENCH]: loading firmware  ../../../sw/build/main.hex
[TESTBENCH]: No Max time specified
[TESTBENCH]: No Boot Option specified, using jtag (boot_sel=0)
[X-HEEP]: NUM_BYTES =        128KB

UART: Created /dev/pts/4 for uart0. Connect to it with any terminal program, e.g.
$ screen /dev/pts/4
UART: Additionally writing all UART output to 'uart0.log'.
Reset Released
Set Exit Loop
Memory Loaded
Simulation finished after 10602 clock cycles
Program Finished with value 0
INFO: Running post_run script print_uart_log
hello world!
```

After the simulation, you can inspect the signals using GTKWave, open with GTKWave the waveform.vcd file located into the sim-verilator folder in the build folder. Then look for testharness and for the data fifo of the UART to see the “hello world!” print. In case you do not have GTKWave installed, run:

```
sudo apt install libcanberra-gtk-module libcanberra-gtk3-module
sudo apt-get install -y gtkwave
```

[2] https://www.veripool.org/verilator/



4.1. Playing with the configs.

To play and see the effect of the different possible configurations, you could execute the following commands:

>>make mcu-gen PYTHON_X_HEEP_CFG=/home/.../GR-HEEP/config/e20_one2M_10B.py X_HEEP_CFG=/home/.../GR-HEEP/hw/vendor/x-heep/configs/python_unsupported.hjson

After having generated the system with the required amount of memory, it is time to simulate using Verilator.

>> make verilator-build

This will create the build folder in root, where the model has been compiled. Then, we can just run the app on it:

>> make verilator-run

Note that there could be different configurations with different results (cycles)

Config	CPU	Bus	Memory	DMA setup	Main intent
e20_one2M_10B.py	CV32E20	ONE2M	NORMAL	1 channel	Baseline reference point
e40p_N2M_2B_INT8B.py	CV32E40P	N2M	INTL	1 channel	Better core + more parallel memory/interconnect behavior

e40p_one2M_10B.py	CV32E40P	ONE2M	NORMAL	1 channel	Isolate core effect while reverting interconnect/memory
e40p_xpulp_N2M_2B_INT8B.py	CV32E40P + XPULP	N2M	INTL	1 channel	Add ISA acceleration on top of a high-throughput fabric

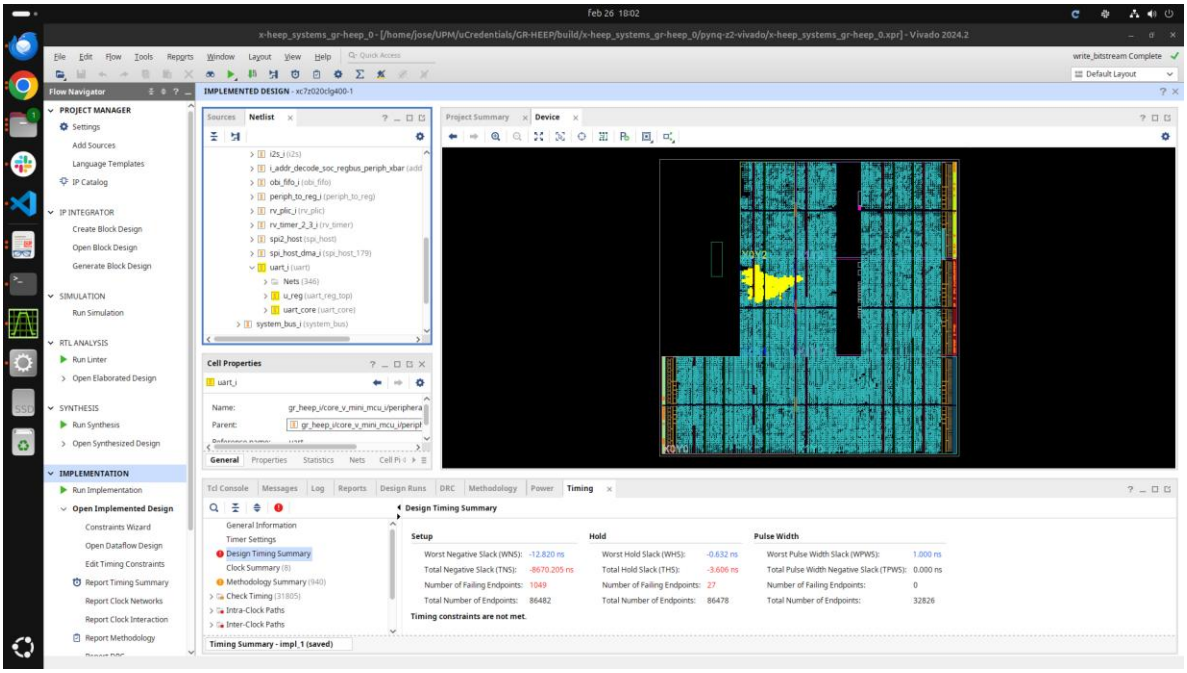
5. Vivado and main X-HEEP hierarchy

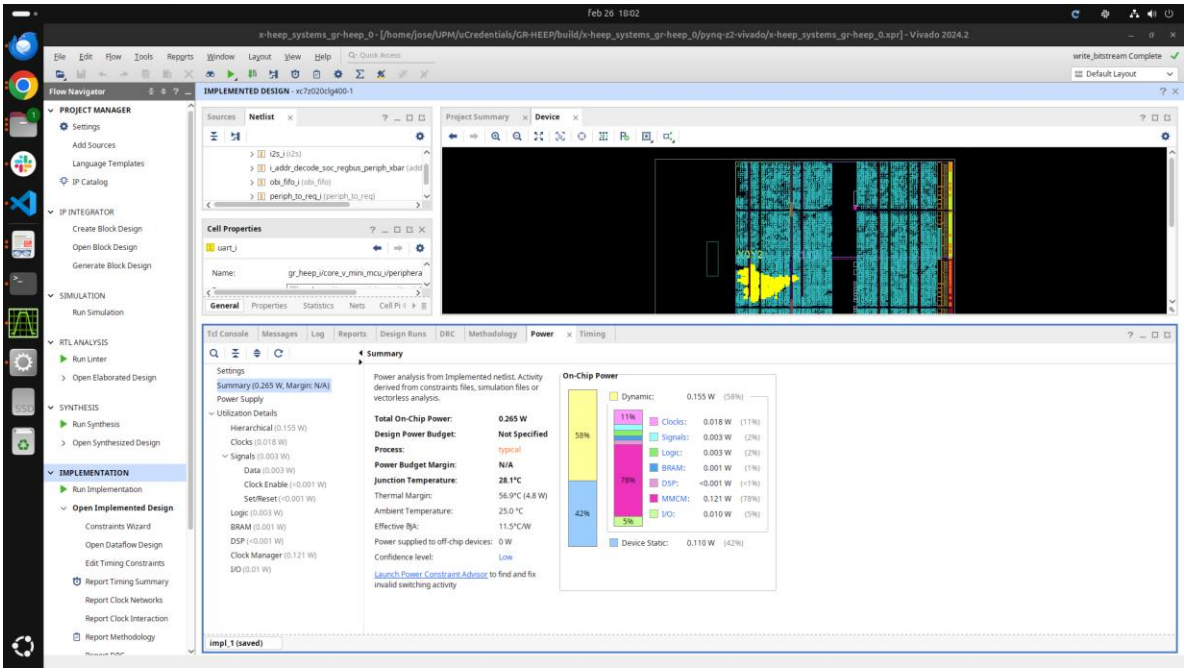
To implement the system and generate the bitstream to load into the FPGA, type:

make vivado-fpga FPGA_BOARD= nexys-a7-100t

This would configure and create a vivado project and a bitstream to prototype into the nexys-a7-100t board. Note that there are more alternatives, such as pynq and zcu variants. In case you do not have capabilities to run the implementation with vivado, please download an already implemented project **here**.

After that, open vivado and explore the hierarchy of the system as well as the estimated power consumption when prototyping into the FPGA:





To implement into the FPGA you can issue the corresponding “vivado-fpga-pgm" command or just open the HW Manager of vivado, select the generated bitstream and program the FPGA. After that, connect the programmer accordingly following what’s specified in *hw/fpga/constraints/nexys/pin_assign.xdc*

6. FPGA Implementation

To be done at the lab.

References

[IEEE’24] IEEE, “IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language”, in *IEEE Std 1800-2023 (Revision of IEEE Std 1800-2017)*, pp. 1–1354, 2024.

Part #2: ADDING A SIMPLE COUNTER PERIPHERAL THROUGH THE XAIF

Prerequisites

This tutorial assumes that you are familiar with the base X-HEEP and have set up and can simulate the execution of basic applications using verilator. For an introduction, follow the instructions available in X-HEEP's documentation[3].

Introduction to the XAIF

The open-source[4] X-HEEP platform [1] allows to plug in accelerators natively to the system bus through the eXtendible Accelerator InterFace (XAIF). The bus exposes a configurable number of slave and master ports to the external XAIF interface so that one or more accelerators with different bandwidth constraints can be connected.

As can be seen in the top left of Figure 1, the XAIF provides enhanced connectivity to external custom accelerators and includes the following ports:

- Slave and master ports: A configurable number of master and slave interfaces based on the OBI bus protocol[5] or on the register interface[6] and connected to the X-HEEP internal system bus, enabling seamless integration of custom-designed accelerators or peripherals.
- Interrupt ports: A configurable set of interrupt lines linked to the X-HEEP internal PLIC interrupt controller, allowing accelerators to notify the host CPU upon operation completion.
- Power control ports: A configurable number of power control interfaces connected to the X-HEEP internal power manager, providing low-power management capabilities for connected accelerators.

Introduction to the Simple Counter Peripheral

In this tutorial, we are going to integrate a simple software-controlled up counter[7] as a peripheral to X-HEEP. The counter implements the OBI bus interface for the counter value and the register interface for the configuration registers. It is controlled using the control and status registers documented in the repository. This simple example will showcase the steps needed to connect more complex accelerators.

[3] https://x-heep.readthedocs.io

[4] https://github.com/x-heep/x-heep

[5] You can read the specifications in https://github.com/openhwgroup/obi.

[6] You can find more information in https://github.com/pulp-platform/register_interface

[7] https://github.com/x-heep/simple_cnt

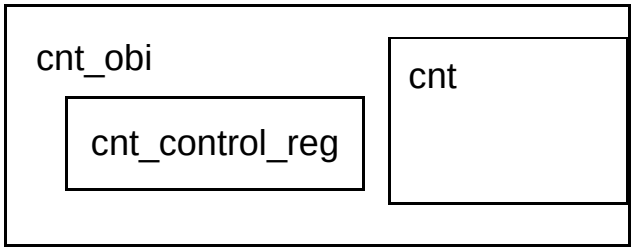


Figure 2: Top-level diagram of the simple counter.

The top-level module for the counter is in hw/cnt.sv, and its OBI bus wrapper is in hw/cnt_obi.sv. This wrapper also contains the control registers (Figure 2). Use a few minutes to get familiar with these modules, especially cnt_obi, as you will later have to connect this to X-HEEP.

eXtending X-HEEP

Until now, we have two different repositories: X-HEEP and the simple counter. Now is the time to put them in the same place. For this, we will have to create a new top-level repository that englobes both. If you already have a repository extending X-HEEP, you can reuse the same one instead of creating a new top level. In this case, skip the next paragraphs until making sure that everything is set up properly and start adding the simple counter.

The procedure of extending X-HEEP is detailed in its documentation[8]. However, since the base X-HEEP platform is meant to be extended, as Figure 1 shows, the open-source[9] GR-HEEP repository already provides a top-level baseline that we can use.

Once you have your GR-HEEP repository, you have to add the counter to it. For this, X-HEEP uses the Vendor tool from OpenTitan[10]. Vendor can import hardware modules and primitives that we want to use inside our project. For more information, you can check the X-HEEP docs[11]. To know where to fetch the files from, Vendor needs a *.vendor.hjson* file like the one you can find for X-HEEP in *hw/vendor/x-heep.vendor.hjson*. Add your own vendor file for the *simple_cnt* based on the one for X-HEEP. You can ignore and delete the patch_dir and the exclude_from_upstream sections. Then, after remembering to activate X-HEEP's conda environment, from the top-level directory run:

```
make vendor-update MODULE_NAME=simple_cnt
```

You should see the simple_cnt project copied into the hw/vendor directory. Now, in the configuration files, we have to add the information on the simple counter. First, add it to the config/mcu-gen-config.py under the slaves memory map in gr_heep_config(). The default offset and length are more than enough.

[8] https://x-heep.readthedocs.io/en/latest/Extending/index.html

[9] https://github.com/x-heep/GR-HEEP

[10] https://opentitan.org/book/util/doc/vendor.html

[11] https://x-heep.readthedocs.io/en/latest/GettingStarted/UnderstandingTools.html

```
ext_xbar_slaves = {
        "simple_cnt": {
                "offset": 0x00000000,
                "length": 0x00010000,
        }
}

Then, add it to the external peripherals.

ext_periph = {
        "simple_cnt": {
                "offset": 0x00000000,
                "length": 0x00001000,
        }
}
```

Finally, set **the external_interrupts to 1.** The next step is instantiating the simple counter module as an external peripheral. This is done in the hw/gr-heep/gr_heep_peripherals.sv.tpl[12]. You can instantiate it and connect it to the proper signals like this:

```
% for a_slave in gr_heep["peripherals"]:
        % if (a_slave['name'] == "SimpleCnt"):
            cnt_obi #(
                    .W (32)
            ) u_cnt_obi (
                    .clk_i(clk_i),
                    .rst_ni(rst_ni),
                    .reg_req_i(gr_heep_peripheral_req[gr_heep_pkg::SimpleCntPeriphIdx]),
                    .reg_rsp_o(gr_heep_peripheral_rsp[gr_heep_pkg::SimpleCntPeriphIdx]),
                    .obi_req_i(gr_heep_slave_req_i[gr_heep_pkg::SimpleCntIdx]),
                    .obi_rsp_o(gr_heep_slave_resp_o[gr_heep_pkg::SimpleCntIdx]),
                    .tc_int_o(gr_heep_peripheral_vec_int[0])
            );
        % endif
% endfor
```

FuseSoC[13] is used in X-HEEP to create a list of hardware modules and interface with electronic design automation (EDA) tools like verilator, questasim, or vivado. To add the simple counter to the project's dependencies, you have to modify the gr-heep.core file. In the

[12] To learn more about .tpl files, you can check the configuration documentation for X-HEEP: https://x-heep.readthedocs.io/en/latest/Configuration/index.html

[13] https://fusesoc.readthedocs.io

dependencies of the files_rtl_generic fileset, add a new line with the name that is defined in the hw/vendor/simple_cnt/cnt.core file (i.e. polito:isa-lab:cnt)[14].

At this point, the connection of the simple counter in hardware should be complete. Verify this by running the hello world with the updated files and fix any errors. Remember to generate the updated files from the templates and configurations with make mcu-gen.

Interacting With and Testing the Simple Counter

Now that the hardware is in place, we have to add the software drivers to interact with the simple counter. For this, you must add three symbolic links in *sw/external/lib/ drivers/simple-cnt* folder to:

- hw/vendor/simple_cnt/sw/cnt_control_reg.h. This will include some definitions for the control registers of the counter.
- hw/vendor/simple_cnt/sw/simple_cnt.h, which is the header file of the driver.
- hw/vendor/simple_cnt/sw/simple_cnt.c, which is the actual implementation of the driver.

Finally, it is time to write an application that will make use of the hardware counter. For that, you have to create a new folder in sw/applications with the name of your application and add a main.c. You can find an example in Appendix A.1.

If needed, increase the number of memory banks of the system by modifying the memory_ss in config/mcu-gen-config.py and re-run make mcu-gen.

```
memory_ss.add_ram_banks([32] * 4)
```

Verify that the simple counter is working properly by running the application and checking the waveforms. Remember, you can check what targets are available in the top-level Makefile by opening it or running make help. You can view the simulation run with Verilator using GTKWave. You will find the generated .fst file in build/.../sim-verilator/ waveform.fst. The design will be in TOP.testharness.gr_heep_i.

Next Steps

Think of some modifications to the simple counter example that would accelerate a simple application and try to implement them. You can start from hw/vendor/simple_ cnt/hw/cnt.sv. Since this is a demo, you can directly modify the RTL of the counter in hw/vendor/simple_cnt. However, keep in mind that this is not the intended way of using the vendor tool. The correct flow in a collaborative environment would be to update the simple_cnt repository, update which revision (commit) we want to point to in hw/vendor/simple_cnt.vendor.hjson, and run again the vendor-update make command to "pull" the changes.

[14] For more details, you can check X-HEEP's documentation on FuseSoC and .core files: https:// x-heep.readthedocs.io/en/latest/GettingStarted/UnderstandingTools.html, or FuseSoC's user guide: https://fusesoc.readthedocs.io/en/stable/user/index.html

Test your modifications to the counter by updating the application that uses it. You can also play around with the application. Remember that you can also modify the drivers in sw/external/lib/drivers/simple_cnt.

References

[1] S. Machetti, P. D. Schiavone, T. C. Müller, M. Peón-Quirós, and D. Atienza, "X-HEEP: An Open-Source, Configurable and Extendible RISC-V Microcontroller for the Exploration of Ultra-Low-Power Edge Accelerators," Jan. 2024.

Appendix A Simple Counter Application

A.1 main.c

```c
// System library headers
#include <stdio.h>

// Custom library headers
#include "core_v_mini_mcu.h"
#include "fast_intr_ctrl.h"
#include "simple_cnt.h"
#include "ext_irq.h"
#include "csr.h"

// Main body
int main(void)
{
    uint32_t val = 255;
    uint32_t ret = 0;

    // Initialize external interrupts handler(s)
    ext_irq_init();

    // Set the counter threshold
    simple_cnt_set_threshold(val);

    // Read back the counter threshold
    ret = simple_cnt_get_threshold();
    if (val != ret) {
        printf("Error: threshold value mismatch\n");
        return -1;
    }
```

```
    // Set the counter value
    val = 42;
    simple_cnt_set_value(val);

    // Read back the counter value
    ret = simple_cnt_get_value();
    if (val != ret) {
        printf("Error: value mismatch\n");
        return -1;
    }

    // Clear the counter
    simple_cnt_clear();

    // Read back the counter value
    ret = simple_cnt_get_value();
    if (0 != ret) {
        printf("Error: value mismatch\n");
        return -1;
    }

    // Enable the counter
    simple_cnt_enable();

    // Wait for the counter to reach the threshold
    simple_cnt_wait_poll();
    printf("TC set\n");

    // Disable the counter and clear it
    simple_cnt_disable();
    simple_cnt_clear();

    // Install the counter interrupt handler and enable the counter
    simple_cnt_irq_install();  // install the handler in X-HEEP's PLIC
    simple_cnt_enable();       // start counting

    // Wait for the counter interrupt
    simple_cnt_wait_irq();     // put CPU to sleep until interrupt is
received
    simple_cnt_disable();      // disable counter to avoid further
interrupts
    printf("IRQ received\n");
```

```
    // Read back the counter value
    ret = simple_cnt_get_value();
    printf("Done. Counter value: %d\n", ret);

    return 0;
}
```

Part #3: ADDING AN XIF-COMPATIBLE COPROCESSOR

1 Introduction to the CV-X-IF in X-HEEP

The open-source[15] X-HEEP platform [1] allows to plug in coprocessors to supported CPU cores through the Core-V eXtension Interface (CV-X-IF)[16]. Here, we will use CV-X-IF and XIF indistinctively, as the second is shorter and can be more convenient. The CV-X-IF-compatible CPU cores in X-HEEP are the cv32e40px, the cv32e40x, and the cv32e20.

As seen in the top right of Figure 1, the CV-X-IF provides a way for coprocessors to be tightly coupled with the CPU. This permits the offloading of instructions that are not recognized by the CPU's decoder to the coprocessor. In this way, new functionality can be added to the CPU without modifying its internal datapath.

Although the CV-X-IF's version at the time of writing is v1.0.0, in this tutorial we will use v0.2.0[17] together with the cv32e40px core. Please take your time to get familiar with the general CV-X-IF and the interfaces of this spec, especially the issue (request/response) interface, the commit interface, and the result interface, as these will be used throughout this tutorial.

2 Introduction to the XIF coprocessor

In this tutorial, we are going to integrate an open-source [18] XIF-compatible example coprocessor to X-HEEP. This coprocessor implements the bit reversal operation in hardware. This example will provide a template and showcase the steps needed to connect more complex coprocessors.

The bit reversal operation *BITREV rd, rs1* follows a custom R-type RISC-V instruction which reverses the bit order of *rs1* and stores the result in *rd*. For example, input *10110000* will result in *00001101*. This operation is needed in Fast Fourier Transform (FFT), cryptography, and error correction codes.

The top-level module of the coprocessor can be found in *hw/xif_copro/xif_copro.sv*, which includes the XIF signals in its ports. The computation of the bit reversal operation is defined in different places:

- The definition of the operation is in *hw/xif_copro/xif_copro_pkg.sv*, in the *copro_op_e* enum.
- The bit-level definition of the RISC-V instruction is in *hw/xif_copro/xif_copro_predecoder_pkg.sv* and *hw/xif_copro/xif_copro_instr_pkg.sv*.
- The decoding takes place in *hw/xif_copro/xif_copro_decoder.sv.*

[15] https://github.com/x-heep/x-heep
[16] https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/ intro.html
[17] https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/v0.2.0/ intro.html
[18] https://github.com/esl-epfl/xif_copro

- The actual bit reversal execution is done in *hw/xif_copro/xif_copro_ex_stage. sv.*

Use a few minutes to get familiar with these modules, as they will be the ones you will need to modify if you want to use this coprocessor as a template to create your own more complex designs.

3 eXtending X-HEEP

Add your own vendor file for the *xif_copro* based on the one for X-HEEP. Again, for reproducibility, we will use a specific commit (*75a9cec*). You can ignore and delete the patch_dir and the exclude_from_upstream sections. Then, after remembering to activate X-HEEP's conda environment, from the top-level directory run:

```
make vendor-update MODULE_NAME=xif_copro
```

You should see the xif_copro project copied into the hw/vendor directory.

The next step is instantiating the coprocessor and connecting it to the microcontroller by attaching the XIF signals. This is done in the *hw/gr_heep.sv.tpl*[19]. Here, have a look at the ext_xif declaration and connections to the microcontroller given by X-HEEP (core_v_mini_mcu). You can instantiate the coprocessor and connect it to the proper signals like this:

```
xif_copro #(
        .XLEN(32),
        .INPUT_BUFFER_DEPTH(1),
        .FORWARDING(1)
) xif_copro_i (
        .clk_i (clk_i),
        .rst_ni(rst_nin_sync),
        .xif_compressed_if(ext_xif.coproc_compressed),
    .xif_issue_if                   (ext_xif.coproc_issue),
    .xif_commit_if                  (ext_xif.coproc_commit),
    .xif_mem_if                     (ext_xif.coproc_mem),
        .xif_mem_result_if          (ext_xif.coproc_mem_result),
    .xif_result_if              (ext_xif.coproc_result)
);
```

FuseSoC[20] is used in X-HEEP to create a list of hardware modules and interface with electronic design automation (EDA) tools like verilator, questasim, or vivado. To add the XIF coprocessor to the project's dependencies, you have to modify the top-level gr-heep.core file.

[19] To learn more about .tpl files, you can check the configuration documentation for X-HEEP: https://x-heep.readthedocs.io/en/latest/Configuration/index.html

[20] https://fusesoc.readthedocs.io

In the dependencies of the files_rtl_generic fileset, add a new line with the name that is defined in the hw/vendor/xif_copro/xif_copro.core file (i.e. esl-epfl:ip:xif_copro)[21].

Then, you can set X-HEEP to use the cv32e40px core by setting it in the config/mcu-gen-config. py configuration like so[22]:

```
from x_heep_gen.cpu.cv32e40px import cv32e40px ...
system.set_cpu(cv32e40px())

#And turn on the use of the CV-X-IF like so:

from x_heep_gen.cv_x_if import CvXIf ...
system.set_xif(CvXIf(
        x_num_rs    = 2,
        x_id_width = 4,
        x_mem_width = 32,
        x_rfr_width = 32,
        x_rfw_width = 32,
            x_misa  = 0x0,
         x_ecs_xs     = 0x0,
    ))
```

Note that the x_num_rs has been set to 2 to align with the requirements of the xif_copro found in xif_copro/hw/if_xif.sv

At this point, the hardware connection of the XIF coprocessor should be complete. Verify this by running the hello world with the updated files and fix any errors. Remember to generate the updated files from the templates and configurations with make mcu-gen.

4 Interacting With and Testing the XIF Coprocessor

Now that the hardware is in place, it is time to write an application that will make use of the XIF coprocessor and its new bit reversal operation. Since the coprocessor is connected directly to the CPU, we don't need external drivers, and we can write the instructions directly in the C code using machine code. For that, you have to create a new folder in sw/applications with the name of your application and add a main.c. You can find an example here or in Appendix A.1.

Verify that the XIF coprocessor is working properly by running the application and checking the results. Remember, you can check what targets are available in the top-level Makefile by opening it or running make help. You can view the simulation run with Verilator using GTKWave.

[21] For more details, you can check X-HEEP's documentation on FuseSoC and .core files: https:// x-heep.readthedocs.io/en/latest/GettingStarted/UnderstandingTools.html, or FuseSoC's user guide: https://fusesoc.readthedocs.io/en/stable/user/index.html

[22] For the full cv32e40px configuration check: https://x-heep.readthedocs.io/en/latest/ Configuration/CPUConfiguration.html

You will find the generated .fst file in *build/.../sim-verilator/ waveform.fst.* The design will be in *TOP.testharness.gr_heep_i.*

5 Next Steps

Implement the bit reversal operation as a software routine and measure the difference in number of cycles between executing it in software and with the hardware version in the coprocessor. You can start from the given application and use the timer_sdk[23] of X-HEEP. Are the results what you expected? If not, try checking the generated assembly code in sw/build/app/main.S. Check the time it takes to execute a NOP (no operation). What do you see in the waveforms? Does it match the assembly and what you expect?

Think of another operation that would be useful to accelerate through the CV-X-IF and implement it in the coprocessor. You can replicate the bit reversal operation by reviewing Section 2. Since this is a demo, you can directly modify the RTL of the coprocessor in hw/vendor/xif_copro. However, keep in mind that this is not the intended way of using the vendor tool. The correct flow in a collaborative environment would be to modify the upstream xif_copro repository, update which revision (commmit) we want to point to in hw/vendor/xif_copro.vendor.hjson, and run again the vendor-update make command to "pull" the changes.

Test your modifications to the coprocessor by modifying the application that is using it. Add a new function to wrap around the machine code of the operation and simplify its use. Does this new instruction effectively accelerate the operation as you initially intended?

References

[1] S. Machetti, P. D. Schiavone, T. C. Müller, M. Peón-Quirós, and D. Atienza, “X-HEEP: An Open-Source, Configurable and Extendible RISC-V Microcontroller for the Exploration of Ultra-Low-Power Edge Accelerators,” Jan. 2024.

[23] You can find the documentation in https://x-heep.readthedocs.io/en/latest/Peripherals/ Timer.html and an example in https://github.com/x-heep/x-heep/blob/main/sw/ applications/example_timer_sdk/main.c

Appendix A XIF Coprocessor Application

A.1 main.c

```
#include <stdio.h>
#include <stdint.h>
/**
*   @brief Performs bit-reversal on a 32-bit integer using a custom RISC-V
instruction.
*
*   This function utilizes an inline assembly instruction to execute a
custom RISC-V * operation (BITREV), which reverses the order of bits in the
input value.
*
*   @param val The 32-bit integer whose bits will be reversed.
*   @return uint32_t The bit-reversed result.
*
*   The `.insn` directive is used to encode a custom RISC-V instruction with
the following format:
*
*   .insn r opcode, funct3, funct7, rd, rs1, rs2
*
*   - `r` specifies that this is an R-type (register-based) instruction.
*   - `0x2B` (0101011) is on of the opcodes for custom RISC-V extensions.
*   - `0x7` is the `funct3` field, which helps define the operation.
*   - `0x02` (00000_10) is the `funct7` field, which further specifies the
BITREV operation.
*   - `%0` (rd) is the destination register where the output (result) is
stored.
*   - `%1` (rs1) is the source register containing the input value (val).
*   - `zero` (rs2) is a hardcoded register operand set to zero, as this
instruction only requires one input regist
*/ static inline uint32_t xbitrev(uint32_t val)
{
 uint32_t result;
 asm volatile (
".insn r 0x2B, 0x7, 0x02, %0, %1, zero" // Custom encoding for BITREV
: "=r"(result) // Output operand (rd)
                : "r"(val)  // Input operand (rs1)
);
  return result;
}
```

```
int main()
{
volatile uint32_t x = 0b10110000;
volatile uint32_t reversed;
reversed = xbitrev(x);
printf("Input: 0x%08X\n", x);
printf("Reversed: 0x%08X\n", reversed);
return 0;
}
```

*Troubleshooting>

If you get any compilation error along the workshop, try to restore the whole path

```
PATH=/home/mfpdsmbaa/tools/riscv/bin:/home/mfpdsmbaa/tools/riscv_lab/riscv/bin:/home/mfpdsmbaa/tools/riscv_lab/bin:/home/mfpdsmbaa/tools/riscv_toolchain_new/bin:/home/mfpdsmbaa/tools/riscv_toolchain_new/bin:/home/mfpdsmbaa/tools/verilator/5.040/bin:/home/mfpdsmbaa/tools/riscv/bin:/home/mfpdsmbaa/tools/verible/v0.0-3622-g07b310a3/bin:/home/mfpdsmbaa/tools/openocd/bin:/home/mfpdsmbaa/miniconda3/envs/core-v-mini-mcu/bin:/home/mfpdsmbaa/miniconda3/condabin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
```